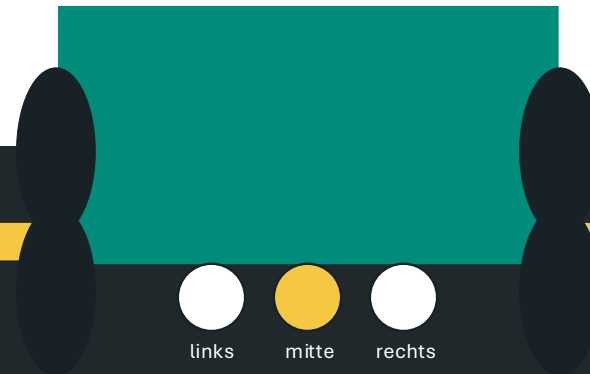


Schwarzfahrer

Linienfolger-Roboter mit PD-Regelung

Team Schwarzfahrer

Nils Lachenmaier; Maik Koren



Team Schwarzfahrer

Ein Linienfolger, der aus drei Sensorsignalen eine stabile Fahrbewegung berechnet.

Nils



Maik

Warum „Schwarzfahrer“?

Unser Roboter fährt zwar nicht ohne Ticket, sondern auf einer schwarzen Linie. Immer dem Schwarzen hinterher.

Projektziel

Saubere Softwarestruktur + schneller PD-Regelkreis + kontrollierte Motorleistung.



Clean Code macht Änderungen kontrollierbar

Drei konkrete Beispiele aus unserer finalen Python-Struktur.

1 Namen statt Zahlen

```
PWM_FREQUENZ_HZ = 100  
GPIO_SENSOR_MITTE = 15  
P_FAKTOR = 10.0
```

Nutzen

Bedeutung und Einstellwerte sind sofort erkennbar und leicht auffindbar.

2 Eine Aufgabe je Modul

```
sensors.py → messen  
control.py → regeln  
motors.py → fahren
```

Nutzen

Fehler lassen sich einem Bereich zuordnen, ohne den gesamten Code zu verändern.

3 Kurze Funktionen

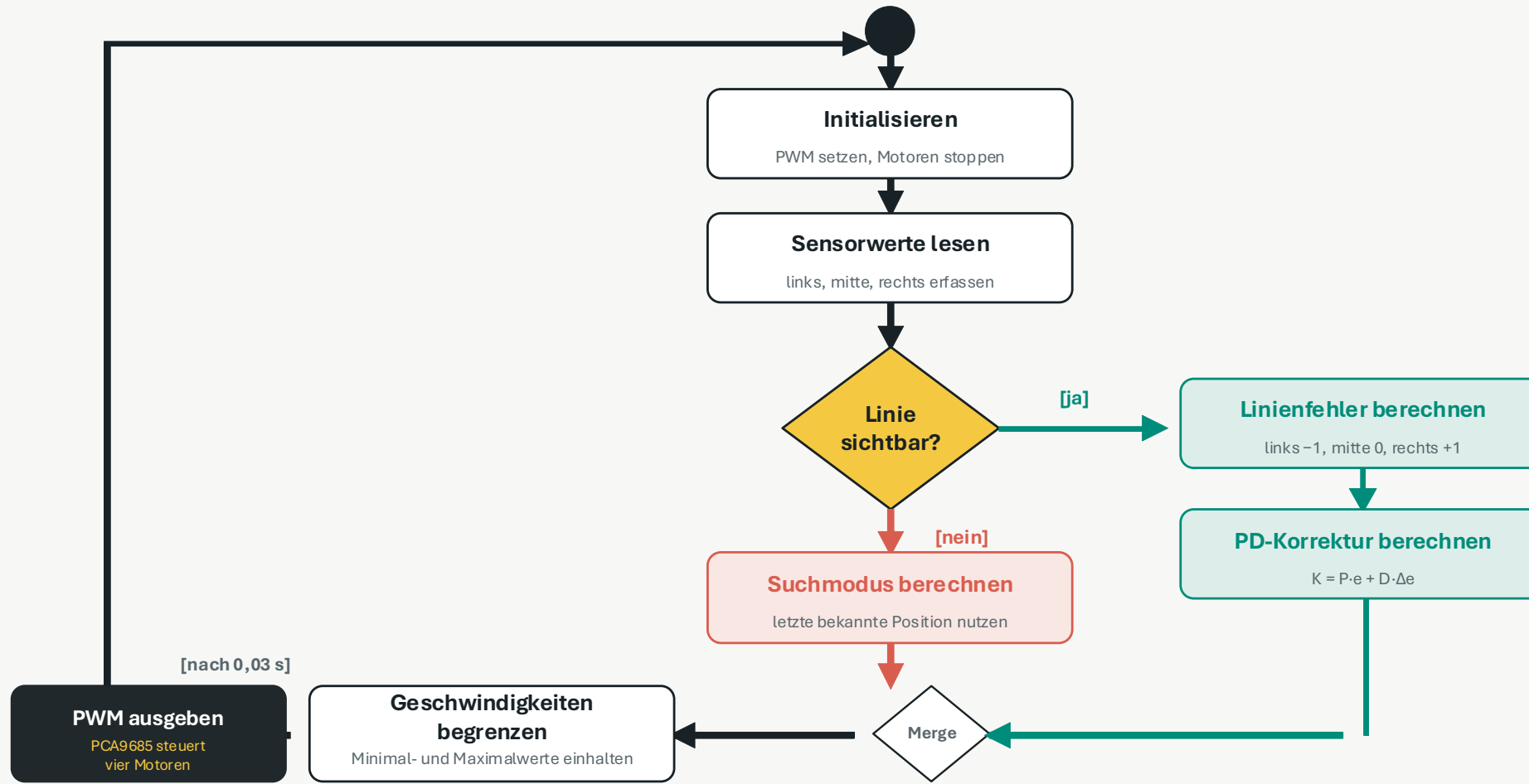
```
_berechne_linienfehler()  
_berechne_korrektur()  
_berechne_suchbefehl()
```

Nutzen

Jede Funktion macht genau eine Sache und kann getrennt verstanden werden.

Linienfolger-Regelkreis

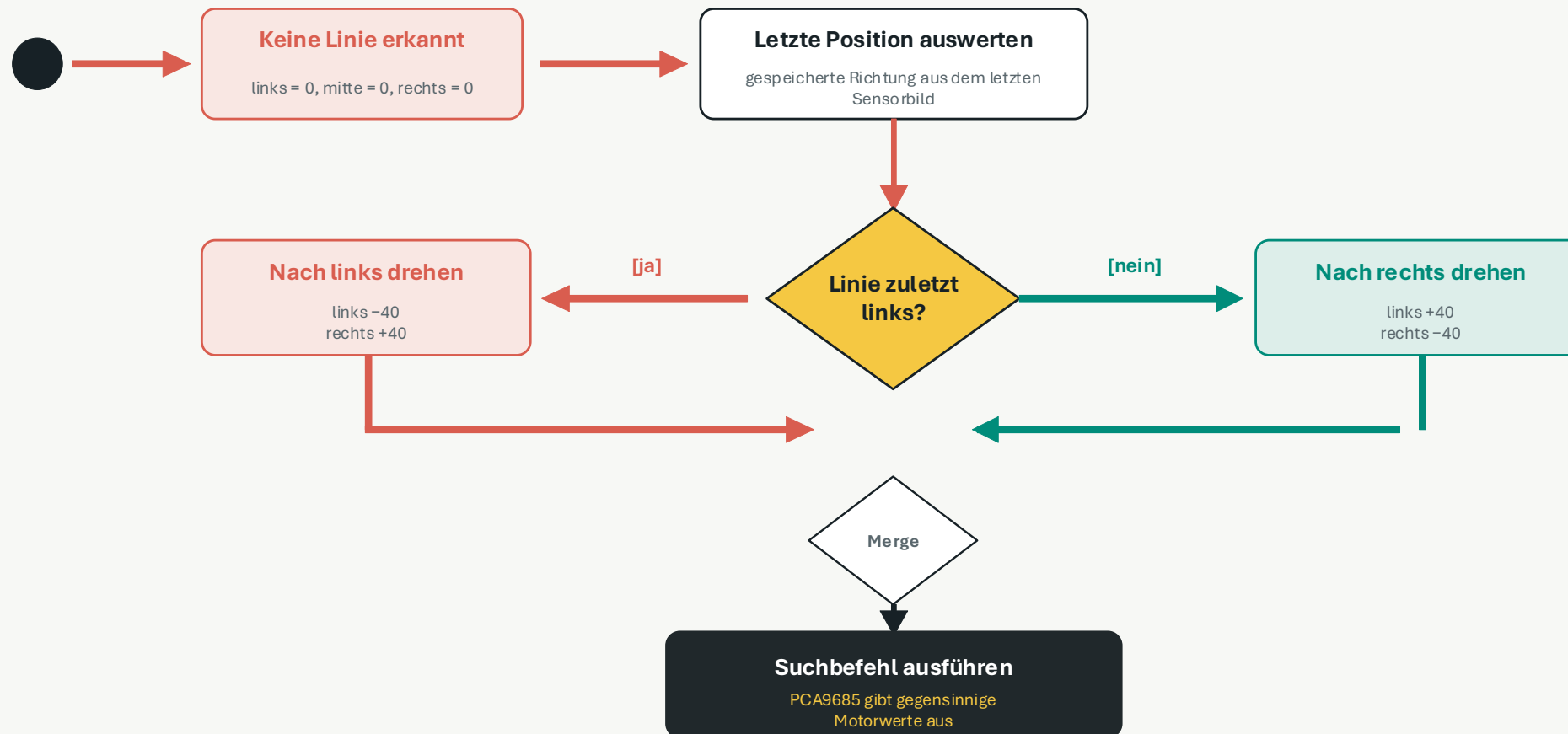
Sauberer Hauptablauf: messen, entscheiden, regeln, Motoren ansteuern und nach 0,03 Sekunden wiederholen.



Legende: schwarzer Kreis = Start · abgerundetes Rechteck = Aktion · Raute = Entscheidung/Merge · [Bedingung] = Guard

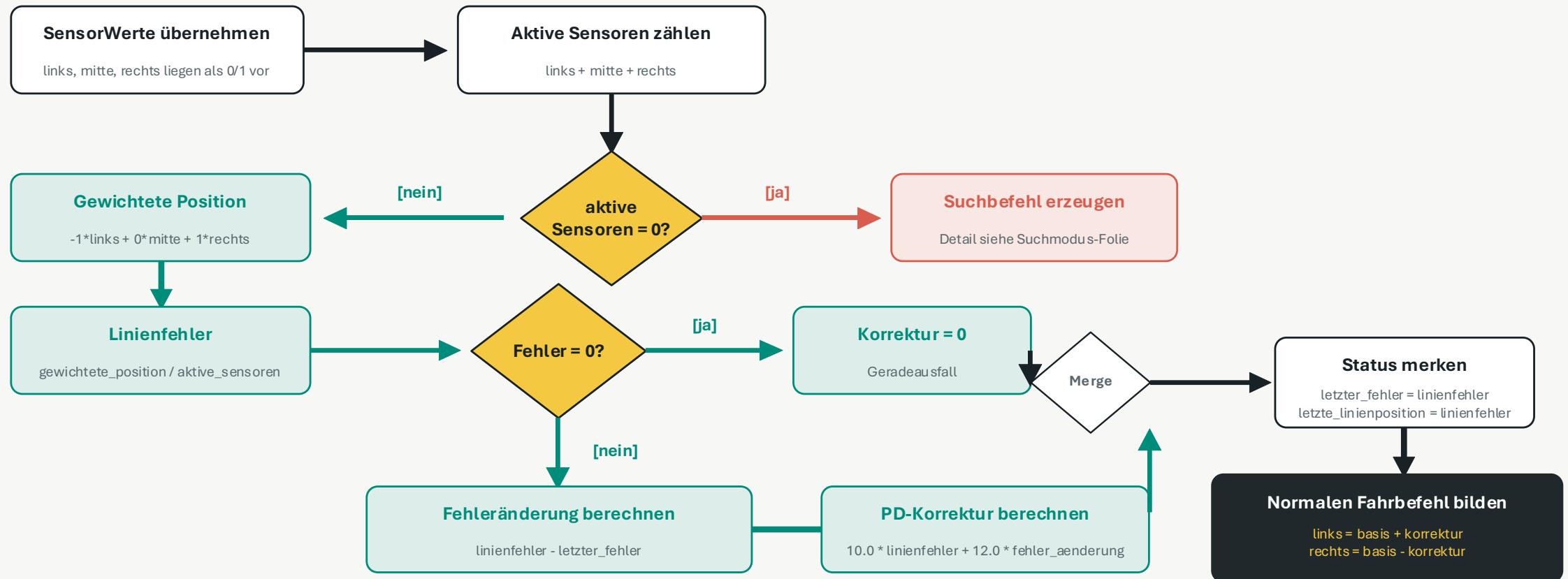
Detail: Suchmodus bei verlorener Linie

Diese Folie erklärt den roten Pfad aus dem Hauptdiagramm genauer.



Detail: Linienfehler und PD-Korrektur

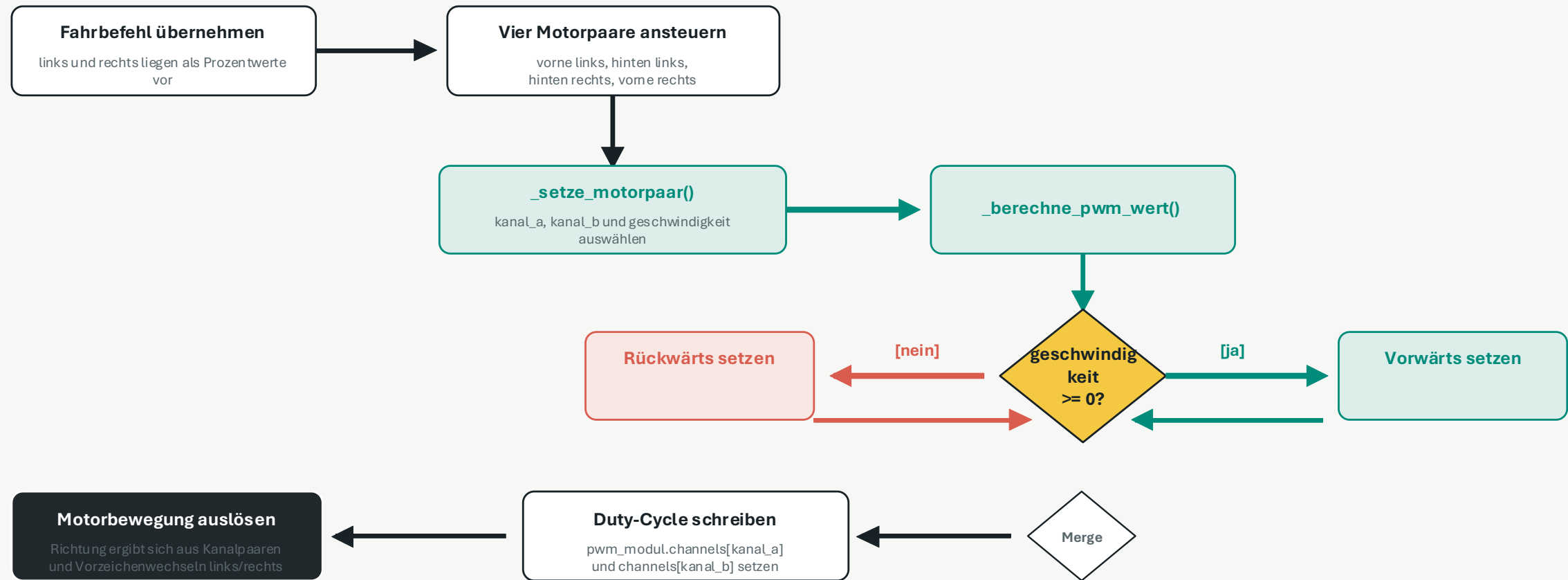
Diese Folie zeigt die eigentliche Rechnung aus ``berechne_fahrbefehl()`` und ``_berechne_korrektur()``.



Die Begrenzung auf Mindest- und Maximalgeschwindigkeit passiert danach in ``_begrenze_fahrgeschwindigkeit()``.

Detail: Motorsteuerung und PWM-Ausgabe

Diese Folie beschreibt ``ausfuehren()`, `_setze_motorpaar()`, und `_berechne_pwm_wert()`, aus `motors.py`.`



Wichtig: In ``ausfuehren()`, werden linke und rechte Seite teilweise mit invertiertem Vorzeichen an die Motorpaare übergeben.`

Warum ein PD-Regler?

Er reagiert schnell, bremst Pendeln und bleibt für drei digitale Sensoren nachvollziehbar.

$$\text{Korrektur} = P \cdot \text{Fehler} + D \cdot \text{Änderung}$$

P-Faktor

reagiert auf die aktuelle
Abweichung

D-Faktor

reagiert auf schnelle
Änderungen

Schnell

Nur wenige Rechenschritte im 0,03-s-Regelzyklus.

Stabil

Dämpft das Hin-und-her-Pendeln auf Geraden.

Robust

Eigener Suchmodus, wenn kein Sensor die Linie sieht.

Nachvollziehbar

Werte lassen sich direkt am Fahrzeug abstimmen.

Ergebnis: klare Softwarestruktur, gezielte Einstellwerte und ein Linienfolger, der Kurven aktiv ausregelt.

Vielen Dank

Team Schwarzfahrer

Linie erkennen. Nachregeln. Ziel finden.
